

Verteilte Systeme, Containerisierung

Arbeitsblatt 07 · Resilienz, Skalierung und Observability

Sie machen den Betrieb sichtbar: Sie lesen Messwerte, untersuchen einen Fehler und testen, wie Kubernetes auf fehlende Bereitschaft und zusätzliche Last reagiert.

Arbeitszeit	Besprechung
105 Minuten	15 Minuten

Am Ende können Sie

- einen Rückstau im Dashboard erkennen und eine Ursache prüfen,
- Readiness, Liveness und Rollback unterscheiden,
- CPU-Requests, Limits und die Reaktion eines HPA erklären.

Material

Voraussetzung: Ihr funktionierendes Projekt nach AB6. Docker Desktop, k3d, kubectl, Helm sind installiert;

- Block 7: [Release v1.1.0](#) / [Kurs-ZIP herunterladen](#)

Laden Sie das angehängte Kurs-ZIP aus Release v1.1.0 herunter, nicht «Source code (zip)». Entpacken Sie es neben Ihrem Projekt. Der Ordner vsc-dispatch-city-07-observability enthält install.ps1 und install.sh; die folgenden Pfade passen dazu.

ORDNERSTRUKTUR

```
Development/
  ContainerProject/          <- hier arbeiten
    vsc-dispatch-city-07-observability/
      install.ps1
      install.sh
```

Vor dem Start

```
kubectl config use-context k3d-teko-k8s
kubectl -n food-delivery get cluster food-delivery-db
kubectl -n food-delivery get pods
```

PostgreSQL und die Anwendung müssen bereit sein. Befehle ohne Shell-Hinweis gelten für PowerShell und Bash. Jeden Befehl einzeln ausführen. Ein Port-Forward bleibt in einem eigenen Terminal offen; weitere Befehle gehören in ein zweites Terminal.

Theorie	Danach arbeiten
Folien 1–5	Aufgaben 1 und 2
Folien 6–8	Aufgaben 3 und 4
Folien 9–12	Aufgabe 5

1

Das Monitoring öffnen

25 Minuten · Vermuten, ausprobieren, erklären

Integrieren Sie das Material mit den Befehlen Ihrer Shell. Der vorbereitete Start baut das Observer-Image, importiert es in k3d, installiert Prometheus/Grafana per Helm und aktiviert die Messpunkte. Rechnen Sie beim ersten Mal mit einigen Minuten Downloadzeit.

WINDOWS POWERSHELL

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
& '..\vsc-dispatch-city-07-observability\install.ps1' -Target '.'
./platform/monitoring/start-course.ps1
```

BASH / MACOS / WSL

```
sh ../vsc-dispatch-city-07-observability/install.sh .
sh platform/monitoring/start-course.sh
```

Öffnen Sie danach Grafana über einen Port-Forward:

```
kubectl -n monitoring port-forward service/monitoring-grafana 3000:80
```

Browser: <http://localhost:3000> · Anmeldung: admin / delivery. Unter Dashboards öffnen Sie «Dispatch City - Betrieb». Wählen Sie «Last 15 minutes» und warten Sie etwa eine Minute auf die ersten Daten.

Öffnen Sie zusätzlich Ihre Stadt über den bekannten Ingress-Zugriff. Als Alternative führt dieser Port-Forward in einem weiteren Terminal ebenfalls über den Ingress-Controller:

```
kubectl -n kube-system port-forward service/traefik 8081:80
```

Stadt: <http://localhost:8081>. Starten Sie die Simulation. Wählen Sie zwei Anzeigen in Grafana aus und beobachten Sie, welche sich verändern.

Sichtbares Resultat: Das Dashboard zeigt Werte für Bestellungen, Pizza-Worker und Warteschlangen.

Welche zwei Anzeigen haben Sie gewählt? Was zeigt die jeweilige Zahl?

Wartende Nachrichten je Restaurant, zeigt was noch in der Queue liegt

Pizza-Worker bereit, wie viele Pizza Pods gerade ready sind

bei mir 0 wartend, Worker 1 von 1

Keine Daten? Erst eine Minute warten und den Zeitraum prüfen. Danach: `kubectl -n monitoring get pods`. «No data» ist nicht dasselbe wie der Messwert 0.

2**Den Rückstau erkennen**

20 Minuten · Vermuten, ausprobieren, erklären

Vermuten Sie zuerst: Was passiert mit der Pizza-Queue, wenn es Bestellungen gibt, aber kein Pizza-Worker mehr läuft? Halten Sie im Grafana-Dashboard die Kurve «Wartende Nachrichten je Restaurant» im Blick.

```
kubect1 -n food-delivery scale deployment/restaurant-pizza --replicas=0
```

Lassen Sie die Simulation etwa eine Minute laufen. Bei wenig Last klicken Sie in der Stadt mehrfach auf «Bestellung»; das Restaurant wird automatisch gewählt. Beobachten Sie gleichzeitig die Anzeige «Pizza-Worker bereit».

Ihre Vermutung und Beobachtung: Welche Kurve oder Zahl verändert sich?

Vermutung: Worker geht auf 0, Pizza Queue steigt

stimmt, Pizza von 0 auf 7, Bowl und Curry bleiben 0, in der Stadt kein Fehler

Bringen Sie den Worker zurück und prüfen Sie, ob er wieder arbeiten kann:

```
kubect1 -n food-delivery scale deployment/restaurant-pizza --replicas=1
kubect1 -n food-delivery rollout status deployment/restaurant-pizza
kubect1 -n food-delivery logs deployment/restaurant-pizza --tail=10
```

Die Warteschlange sollte sich nach und nach leeren. Falls sie weiter wächst: Sind neue Bestellungen schneller als die Verarbeitung? Ist der Worker bereit? Wählen Sie eine dieser Fragen und prüfen Sie sie.

Sichtbares Resultat: Sie sehen Rückstau und anschliessend seine Auflösung im Zeitverlauf.

Was wäre hier die erste sinnvolle Massnahme? Begründen Sie mit einer Beobachtung.

Pizza-Worker wieder auf 1 und schauen ob er bereit ist, nicht einfach mehr Replicas

Worker bereit war 0 und nur Pizza ist gestiegen, also ein Ausfall

danach ging die Queue wieder auf 0

Wichtig: replicas=0 ist ein absichtlich geänderter Soll-Zustand. Kubernetes startet den Worker deshalb nicht von selbst neu. Stellen Sie am Ende wieder replicas=1 her.

3

Readiness sichtbar testen

18 Minuten · Vermuten, ausprobieren, erklären

Die folgenden Aufgaben nutzen eine kleine NGINX-Anwendung im eigenen Namespace betrieb-lab. So bleibt der Versuch überschaubar. Zwei Replicas teilen sich einen Service.

```
kubectl apply -f labs/block-07/web.yaml
kubectl -n betrieb-lab rollout status deployment/lab-web
kubectl -n betrieb-lab get pods
```

Kopieren Sie einen Pod-Namen. Ersetzen Sie PODNAME in beiden exec-Befehlen durch genau diesen Namen. Die Readiness-Probe prüft im Lab absichtlich nur, ob /tmp/ready existiert.

```
kubectl -n betrieb-lab exec PODNAME -- mv /tmp/ready /tmp/not-ready
kubectl -n betrieb-lab get pods
kubectl -n betrieb-lab get endpointslice -o yaml
```

Warten Sie fünf Sekunden und wiederholen Sie die beiden get-Befehle. Suchen Sie in der EndpointSlice unter endpoints die Felder targetRef.name und conditions.ready. Vergleichen Sie diese mit READY und RESTARTS des betroffenen Pods.

Wird der Container neu gestartet? Woran erkennen Sie, ob der Service ihn noch verwendet?

Nein, RESTARTS bleibt 0, nur READY 0/1

in der EndpointSlice ready: false, bekommt keinen Traffic mehr

Stellen Sie die Bereitschaft wieder her:

```
kubectl -n betrieb-lab exec PODNAME -- mv /tmp/not-ready /tmp/ready
kubectl -n betrieb-lab get pods
```

Sichtbares Resultat: Ein Pod wechselt auf 0/1 und wieder auf 1/1. Sein Restart-Zähler bleibt gleich.

In einer echten Anwendung prüft Readiness, ob Anfragen verarbeitet werden können. Die Datei ist nur unser einfacher Lab-Schalter.

4**Ein Update stoppen und zurückrollen**

17 Minuten · Vermuten, ausprobieren, erklären

Öffnen Sie die Testseite in einem separaten Terminal:

```
kubectl -n betriebs-lab port-forward service/lab-web 8088:80
```

Browser: <http://localhost:8088>. Öffnen Sie `labs/block-07/web.yaml`. Suchen Sie `maxUnavailable: 0` und `maxSurge: 1`. Vermuten Sie, was bei einem unbrauchbaren neuen Image passiert.

Starten Sie nun bewusst ein Update mit einem Image-Tag, der nicht existiert:

```
kubectl -n betriebs-lab set image deployment/lab-web web=nginx:absichtlich-falsch
kubectl -n betriebs-lab get pods
kubectl -n betriebs-lab describe deployment lab-web
```

Nach etwa 30–60 Sekunden erscheint beim neuen Pod `ErrImagePull` oder `ImagePullBackOff`. Kopieren Sie seinen Namen und lesen Sie die Kubernetes Events:

```
kubectl -n betriebs-lab describe pod PODNAME
```

Welche Meldung erklärt den Fehler? Warum bleiben die alten Pods verfügbar?

Failed to pull image `nginx:absichtlich-falsch`: not found, `ImagePullBackOff`

`maxUnavailable 0`, die alten bleiben bis ein neuer ready ist

Rollen Sie das Update zurück:

```
kubectl -n betriebs-lab rollout undo deployment/lab-web
kubectl -n betriebs-lab rollout status deployment/lab-web
```

Sichtbares Resultat: Die vorherige Version läuft wieder mit zwei bereiten Pods.

Der YAML-Stand blieb hier korrekt und enthält den gültigen Tag. Ein Rollback setzt die Pod-Vorlage zurück, nicht den Inhalt einer Datenbank. Der Port-Forward zeigt eine einzelne Pod-Verbindung und beweist allein keine unterbrechungsfreie Lastverteilung.

5

Ressourcen und automatische Skalierung

25 Minuten · Vermuten, ausprobieren, erklären

Öffnen Sie labs/block-07/web.yaml. Jeder Container hat CPU-Request 100m und CPU-Limit 200m. Prüfen Sie den aktuellen Verbrauch:

```
kubectl -n betriebs-lab top pods
```

Wie viel CPU fordert ein Pod an? Wie viel darf er höchstens nutzen? Was sind 100m in CPU-Kernen?

Request 100m, Limit 200m, 100m = 0,1 Kern

top pods im Leerlauf ca. 2-3m

Öffnen Sie labs/block-07/hpa.yaml. Erhöhen Sie maxReplicas von 3 auf 4. Belassen Sie minReplicas: 2 und das CPU-Ziel 50. Aktivieren Sie den HPA und beobachten Sie ihn in Terminal A:

```
kubectl apply -f labs/block-07/hpa.yaml  
kubectl -n betriebs-lab get hpa -w
```

Starten Sie in Terminal B den passenden Lasttest. Er erzeugt 150 Sekunden künstliche CPU-Last in einem Pod und beendet sich selbst.

BASH / MACOS / WSL

```
sh labs/block-07/load.sh
```

POWERSHELL · FREIGABE GILT PRO TERMINAL

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass  
./labs/block-07/load.ps1
```

Prüfen Sie in einem weiteren Terminal mit get pods, ob Replicas hinzukommen. Der HPA braucht Messwerte und etwas Zeit; das Hoch- und Herunterskalieren ist nicht sofort sichtbar. <unknown> direkt nach dem Start kann normal sein.

Sichtbares Resultat: Der HPA erhält CPU-Messwerte und erhöht unter Last die Replica-Zahl innerhalb Ihrer Grenze.

Welche Replica-Zahlen haben Sie gesehen? Warum kann eine Queue trotz niedriger CPU wachsen?

2, dann 3, dann 4

Worker wartet z.B. auf die DB, braucht kaum CPU, die Queue wächst trotzdem

Beenden Sie die Beobachtung mit Ctrl+C. Warten Sie nach dem Lasttest einige Minuten auf die Rückkehr zu zwei Pods. Falls <unknown> bleibt: kubectl top pods und kubectl describe hpa lab-web im Namespace betriebs-lab prüfen. Auf einem stark ausgelasteten Laptop den Versuch gemeinsam besprechen.